

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option('display.max_columns', None)
```

```
orders = pd.read_excel('/content/List of Orders.xlsx')
order_details = pd.read_excel('/content/Order Details.xlsx')
sales_target = pd.read_excel('/content/Sales target.xlsx')
```

```
print("List of Orders")
display(orders.head())

print("\nShape:", orders.shape)

print("\nColumns:")
print(list(orders.columns))

print("\nInfo:")
orders.info()
```

List of Orders

	Order ID	Order Date	CustomerName	State	City
0	B-25601	2018-01-04 00:00:00	Bharat	Gujarat	Ahmedabad
1	B-25602	2018-01-04 00:00:00	Pearl	Maharashtra	Pune
2	B-25603	2018-03-04 00:00:00	Jahan	Madhya Pradesh	Bhopal
3	B-25604	2018-03-04 00:00:00	Divsha	Rajasthan	Jaipur
4	B-25605	2018-05-04 00:00:00	Kasheen	West Bengal	Kolkata

Shape: (500, 5)

Columns:
['Order ID', 'Order Date', 'CustomerName', 'State', 'City']

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
Column Non-Null Count Dtype
--- ---
0 Order ID 500 non-null object
1 Order Date 500 non-null object
2 CustomerName 500 non-null object
3 State 500 non-null object
4 City 500 non-null object
dtypes: object(5)
memory usage: 19.7+ KB

```
print("Order Details")
display(order_details.head())

print("\nShape:", order_details.shape)

print("\nColumns:")
print(order_details.columns)

print("\nInfo:")
order_details.info()
```

Order Details

	Order ID	Amount	Profit	Quantity	Category	Sub-Category
0	B-25601	1275	-1148	7	Furniture	Bookcases
1	B-25601	66	-12	5	Clothing	Stole
2	B-25601	8	-2	3	Clothing	Hankerchief
3	B-25601	80	-56	4	Electronics	Electronic Games
4	B-25602	168	-111	2	Electronics	Phones

Shape: (1500, 6)

Columns:

Index(['Order ID', 'Amount', 'Profit', 'Quantity', 'Category', 'Sub-Category'], dtype='object')

Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1500 entries, 0 to 1499

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Order ID	1500 non-null	object
1	Amount	1500 non-null	int64
2	Profit	1500 non-null	int64
3	Quantity	1500 non-null	int64
4	Category	1500 non-null	object
5	Sub-Category	1500 non-null	object

dtypes: int64(3), object(3)

memory usage: 70.4+ KB

```
print("Sales Target")
display(sales_target.head())

print("\nShape:", sales_target.shape)

print("\nColumns:")
print(sales_target.columns)

print("\nInfo:")
sales_target.info()
```

Sales Target

	Month of Order Date	Category	Target
0	2026-04-18	Furniture	10400
1	2026-05-18	Furniture	10500
2	2026-06-18	Furniture	10600
3	2026-07-18	Furniture	10800
4	2026-08-18	Furniture	10900

Shape: (36, 3)

Columns:

Index(['Month of Order Date', 'Category', 'Target'], dtype='object')

Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 36 entries, 0 to 35

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Month of Order Date	36 non-null	datetime64[ns]
1	Category	36 non-null	object
2	Target	36 non-null	int64

dtypes: datetime64[ns](1), int64(1), object(1)

memory usage: 996.0+ bytes

```
print("Missing Values in List of Orders")
display(orders.isnull().sum())

print("\nMissing Values in Order Details")
display(order_details.isnull().sum())

print("\nMissing Values in Sales Target")
display(sales_target.isnull().sum())
```

Missing Values in List of Orders

	0
Order ID	0
Order Date	0
CustomerName	0
State	0
City	0

dtype: int64

Missing Values in Order Details

	0
Order ID	0
Amount	0
Profit	0
Quantity	0
Category	0
Sub-Category	0

dtype: int64

Missing Values in Sales Target

	0
Month of Order Date	0
Category	0
Target	0

dtype: int64

```
print("Duplicate Rows in List of Orders:", orders.duplicated().sum())
print("Duplicate Rows in Order Details:", order_details.duplicated().sum())
print("Duplicate Rows in Sales Target:", sales_target.duplicated().sum())
```

```
Duplicate Rows in List of Orders: 0
Duplicate Rows in Order Details: 0
Duplicate Rows in Sales Target: 0
```

```
orders["Order Date"] = pd.to_datetime(orders["Order Date"])
```

```
orders.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Order ID        500 non-null   object
1   Order Date      500 non-null   datetime64[ns]
2   CustomerName    500 non-null   object
3   State           500 non-null   object
4   City            500 non-null   object
dtypes: datetime64[ns](1), object(4)
memory usage: 19.7+ KB
```

```
merged_data = pd.merge(orders, order_details, on="Order ID")
```

```
print("Merged Dataset")
display(merged_data.head())

print("\nShape:", merged_data.shape)

print("\nColumns:")
print(merged_data.columns)
```

Merged Dataset

	Order ID	Order Date	CustomerName	State	City	Amount	Profit	Quantity	Category	Sub-Category
0	B-25601	2018-01-04	Bharat	Gujarat	Ahmedabad	1275	-1148	7	Furniture	Bookcases
1	B-25601	2018-01-04	Bharat	Gujarat	Ahmedabad	66	-12	5	Clothing	Stole
2	B-25601	2018-01-04	Bharat	Gujarat	Ahmedabad	8	-2	3	Clothing	Hankerchief
3	B-25601	2018-01-04	Bharat	Gujarat	Ahmedabad	80	-56	4	Electronics	Electronic Games
4	B-25602	2018-01-04	Pearl	Maharashtra	Pune	168	-111	2	Electronics	Phones

Shape: (1500, 10)

Columns:

```
Index(['Order ID', 'Order Date', 'CustomerName', 'State', 'City', 'Amount',
      'Profit', 'Quantity', 'Category', 'Sub-Category'],
      dtype='object')
```

```
category_sales = (
    merged_data.groupby("Category")["Amount"]
    .sum()
    .reset_index()
    .sort_values(by="Amount", ascending=False)
)
```

```
print("Total Sales by Category")
display(category_sales)
```

Total Sales by Category

	Category	Amount
1	Electronics	165267
0	Clothing	139054
2	Furniture	127181

```
order_profit = (
    merged_data.groupby(["Category", "Order ID"])["Profit"]
    .sum()
    .reset_index()
)
```

```
average_profit = (
    order_profit.groupby("Category")["Profit"]
    .mean()
    .reset_index(name="Average Profit per Order")
)
```

```
print("Average Profit per Order by Category")
display(average_profit)
```

Average Profit per Order by Category

	Category	Average Profit per Order
0	Clothing	28.404580
1	Electronics	51.441176
2	Furniture	12.354839

```
profit_margin = (
    merged_data.groupby("Category")[["Profit", "Amount"]]
    .sum()
    .reset_index()
)

profit_margin["Profit Margin (%)"] = (
    profit_margin["Profit"] / profit_margin["Amount"]
) * 100

profit_margin = profit_margin[["Category", "Profit Margin (%)"]]

print("Profit Margin by Category")
display(profit_margin)
```

Profit Margin by Category

	Category	Profit Margin (%)
0	Clothing	8.027817
1	Electronics	6.349725
2	Furniture	1.806874

```
category_summary = category_sales.merge(average_profit, on="Category")
category_summary = category_summary.merge(profit_margin, on="Category")
```

```
print("Category Performance Summary")
display(category_summary)
```

Category Performance Summary

	Category	Amount	Average Profit per Order	Profit Margin (%)
0	Electronics	165267	51.441176	6.349725
1	Clothing	139054	28.404580	8.027817
2	Furniture	127181	12.354839	1.806874

```
top_sales = category_summary.loc[category_summary["Amount"].idxmax()]
top_profit = category_summary.loc[category_summary["Average Profit per Order"].idxmax()]
top_margin = category_summary.loc[category_summary["Profit Margin (%)"].idxmax()]
```

```
print("Highest Sales Category")
display(top_sales)
```

```
print("Highest Average Profit per Order")
display(top_profit)
```

```
print("Highest Profit Margin")
display(top_margin)
```

Highest Sales Category

	0
Category	Electronics
Amount	165267
Average Profit per Order	51.441176
Profit Margin (%)	6.349725

```
dtype: object
Highest Average Profit per Order
```

	0
Category	Electronics
Amount	165267
Average Profit per Order	51.441176
Profit Margin (%)	6.349725

```
dtype: object
Highest Profit Margin
```

	1
Category	Clothing
Amount	139054
Average Profit per Order	28.40458
Profit Margin (%)	8.027817

```
dtype: object
```

```
low_sales = category_summary.loc[category_summary["Amount"].idxmin()]
low_profit = category_summary.loc[category_summary["Average Profit per Order"].idxmin()]
low_margin = category_summary.loc[category_summary["Profit Margin (%)"].idxmin()]
```

```
print("Lowest Sales Category")
display(low_sales)
```

```
print("Lowest Average Profit per Order")
display(low_profit)
```

```
print("Lowest Profit Margin")
display(low_margin)
```

Lowest Sales Category

2

Category	Furniture
Amount	127181
Average Profit per Order	12.354839
Profit Margin (%)	1.806874

dtype: object

Lowest Average Profit per Order

2

Category	Furniture
Amount	127181
Average Profit per Order	12.354839
Profit Margin (%)	1.806874

dtype: object

Lowest Profit Margin

2

Category	Furniture
Amount	127181
Average Profit per Order	12.354839
Profit Margin (%)	1.806874

dtype: object

```
category_summary = (
    category_sales
    .merge(average_profit, on="Category")
    .merge(profit_margin, on="Category")
)
```

```
print("Category Performance Summary")
display(category_summary)
```

Category Performance Summary

	Category	Amount	Average Profit per Order	Profit Margin (%)
0	Electronics	165267	51.441176	6.349725
1	Clothing	139054	28.404580	8.027817
2	Furniture	127181	12.354839	1.806874

BUSINESS INSIGHTS

Electronics proved the most lucrative, both in total revenue and average profit per purchase, pointing to strong interest from buyers along with higher earnings.

clothing demonstrated the greatest profit margin, illustrating greater income compared to sales.

Furniture exhibited the fewest sales, smallest average profit per transaction, yet smallest profit margin, standing out as the least effective section.

The poor results seen in Furniture could result from elevated overhead expenses, reduced consumer interest, or issues with pricing.

Adjusting ads, streamlining stock levels, assessing costs on Furniture items - such measures might strengthen its financial outcome.

✓ Part 2: Target Achievement Analysis

```
print("Sales Target Dataset")
display(sales_target.head())

print("\nShape:", sales_target.shape)
```

```
print("\nColumns:")
print(list(sales_target.columns))
```

Sales Target Dataset

	Month of Order Date	Category	Target
0	2026-04-18	Furniture	10400
1	2026-05-18	Furniture	10500
2	2026-06-18	Furniture	10600
3	2026-07-18	Furniture	10800
4	2026-08-18	Furniture	10900

Shape: (36, 3)

Columns:
['Month of Order Date', 'Category', 'Target']

```
furniture_target = sales_target[sales_target["Category"] == "Furniture"].copy()
```

```
print("Furniture Sales Target")
display(furniture_target)
```

Furniture Sales Target

	Month of Order Date	Category	Target
0	2026-04-18	Furniture	10400
1	2026-05-18	Furniture	10500
2	2026-06-18	Furniture	10600
3	2026-07-18	Furniture	10800
4	2026-08-18	Furniture	10900
5	2026-09-18	Furniture	11000
6	2026-10-18	Furniture	11100
7	2026-11-18	Furniture	11300
8	2026-12-18	Furniture	11400
9	2026-01-19	Furniture	11500
10	2026-02-19	Furniture	11600
11	2026-03-19	Furniture	11800

```
print(furniture_target.dtypes)
```

```
Month of Order Date    datetime64[ns]
Category               object
Target                int64
dtype: object
```

```
furniture_target["Target Change (%)"] = (
    furniture_target["Target"].pct_change() * 100
)
```

```
print("Furniture Target Analysis")
display(furniture_target)
```

Furniture Target Analysis

	Month of Order Date	Category	Target	Target Change (%)
0	2026-04-18	Furniture	10400	NaN
1	2026-05-18	Furniture	10500	0.961538
2	2026-06-18	Furniture	10600	0.952381
3	2026-07-18	Furniture	10800	1.886792
4	2026-08-18	Furniture	10900	0.925926
5	2026-09-18	Furniture	11000	0.917431
6	2026-10-18	Furniture	11100	0.909091
7	2026-11-18	Furniture	11300	1.801802
8	2026-12-18	Furniture	11400	0.884956
9	2026-01-19	Furniture	11500	0.877193
10	2026-02-19	Furniture	11600	0.869565
11	2026-03-19	Furniture	11800	1.724138

```
furniture_target["Target Change (%)"] = (
    furniture_target["Target Change (%)"].round(2)
)

display(furniture_target)
```

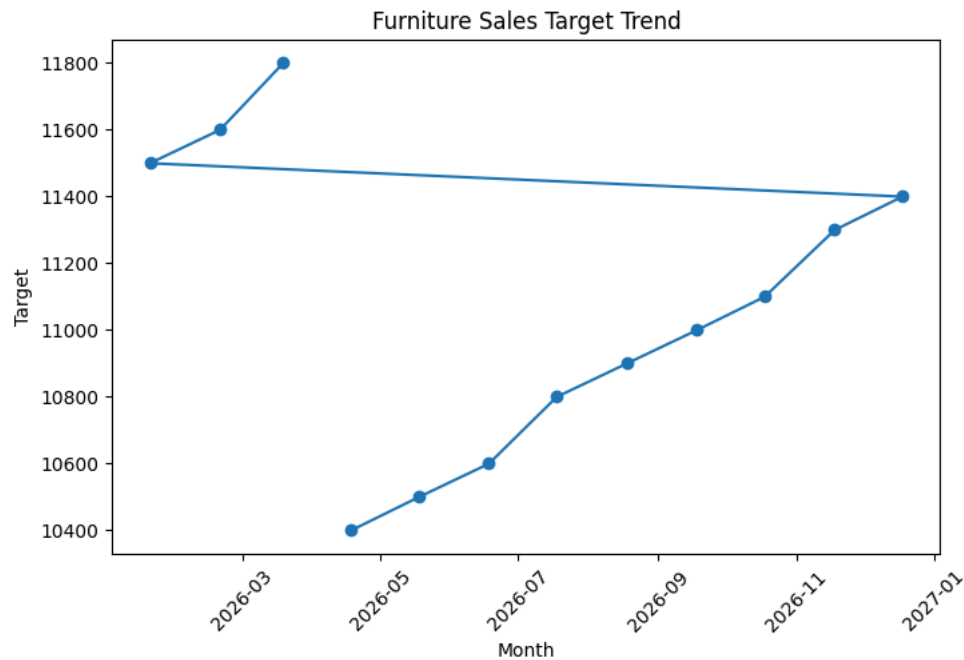
	Month of Order Date	Category	Target	Target Change (%)
0	2026-04-18	Furniture	10400	NaN
1	2026-05-18	Furniture	10500	0.96
2	2026-06-18	Furniture	10600	0.95
3	2026-07-18	Furniture	10800	1.89
4	2026-08-18	Furniture	10900	0.93
5	2026-09-18	Furniture	11000	0.92
6	2026-10-18	Furniture	11100	0.91
7	2026-11-18	Furniture	11300	1.80
8	2026-12-18	Furniture	11400	0.88
9	2026-01-19	Furniture	11500	0.88
10	2026-02-19	Furniture	11600	0.87
11	2026-03-19	Furniture	11800	1.72

```
plt.figure(figsize=(8,5))

plt.plot(
    furniture_target["Month of Order Date"],
    furniture_target["Target"],
    marker="o"
)

plt.title("Furniture Sales Target Trend")
plt.xlabel("Month")
plt.ylabel("Target")
plt.xticks(rotation=45)

plt.show()
```

```
highest_change = furniture_target.loc[
    furniture_target["Target Change (%)"].idxmax()
]

lowest_change = furniture_target.loc[
    furniture_target["Target Change (%)"].idxmin()
]

print("Highest Monthly Increase")
display(highest_change)

print("Lowest Monthly Increase")
display(lowest_change)
```

Highest Monthly Increase

	3
Month of Order Date	2026-07-18 00:00:00
Category	Furniture
Target	10800
Target Change (%)	1.89

dtype: object

Lowest Monthly Increase

	10
Month of Order Date	2026-02-19 00:00:00
Category	Furniture
Target	11600
Target Change (%)	0.87

dtype: object

Target Achievement Insights

- Sales targets for furniture have been steadily increasing throughout the year.
- The monthly increases in targets are mostly small and gradual, showing a steady approach to growth.
- The highest month-over-month increase was observed in July 2018 (1.89%), while the smallest increase was observed in February 2019 (0.87%), indicating that the target growth remained stable throughout the year.
- There have been no big drops in targets, which shows confidence in the furniture category.
- Keeping track of real sales compared to the set targets can help make better plans and use resources more effectively.

Question 1 - Part 3: Regional Performance Insights

```
state_orders = (
    orders.groupby("State")["Order ID"]
    .nunique()
    .reset_index(name="Order Count")
    .sort_values(by="Order Count", ascending=False)
)

top_states = state_orders.head(5)

print("Top 5 States by Order Count")
display(top_states)
```

Top 5 States by Order Count

	State	Order Count
10	Madhya Pradesh	101
11	Maharashtra	90
14	Rajasthan	32
4	Gujarat	27
13	Punjab	25

```
top_state_data = merged_data[
    merged_data["State"].isin(top_states["State"])
]

regional_summary = (
    top_state_data.groupby("State")
    .agg(
        Total_Sales=("Amount", "sum"),
        Average_Profit=("Profit", "mean")
    )
    .reset_index()
)

regional_summary = regional_summary.merge(top_states, on="State")

regional_summary = regional_summary[
    ["State", "Order Count", "Total_Sales", "Average_Profit"]
]

regional_summary = regional_summary.sort_values(
    by="Order Count",
    ascending=False
)

print("Regional Performance Summary")
display(regional_summary)
```

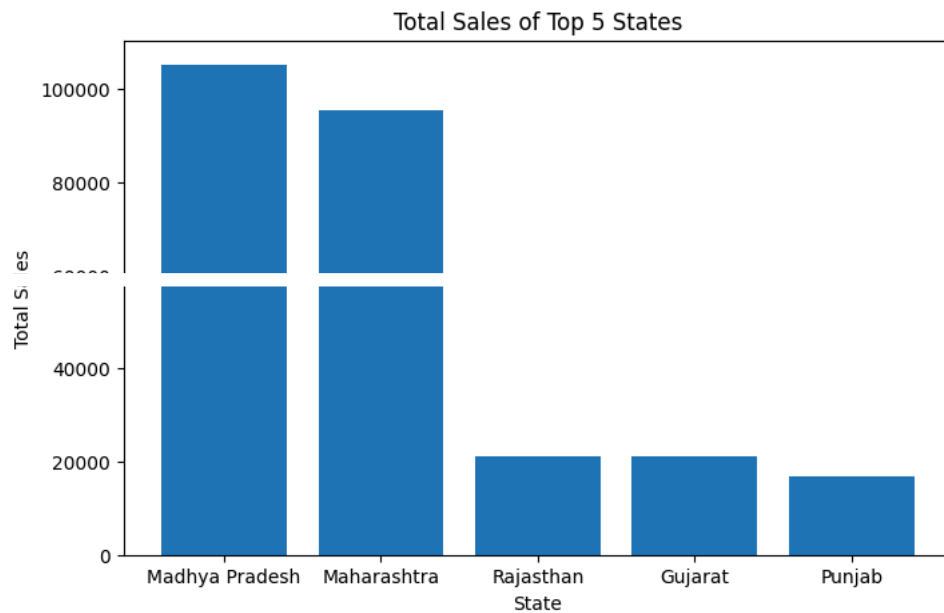
Regional Performance Summary

	State	Order Count	Total_Sales	Average_Profit
1	Madhya Pradesh	101	105140	16.326471
2	Maharashtra	90	95348	21.296552
4	Rajasthan	32	21149	16.986486
0	Gujarat	27	21058	5.344828
3	Punjab	25	16786	-10.150000

```
plt.figure(figsize=(8,5))

plt.bar(
    regional_summary["State"],
    regional_summary["Total_Sales"]
)

plt.title("Total Sales of Top 5 States")
plt.xlabel("State")
plt.ylabel("Total Sales")
```

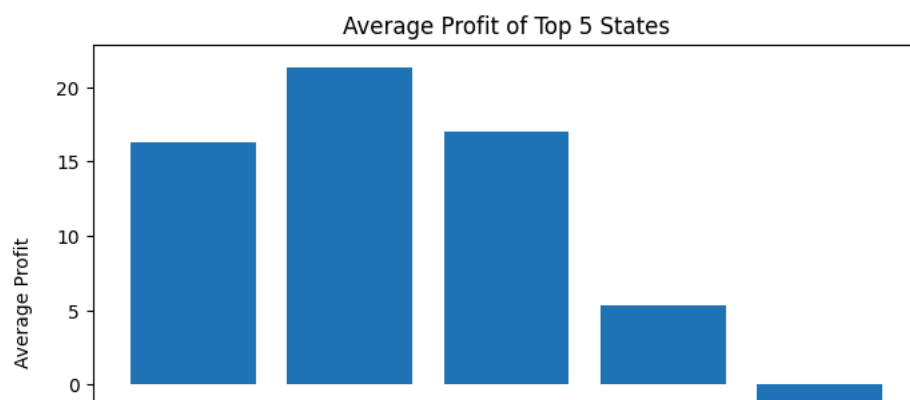


```
plt.figure(figsize=(8,5))

plt.bar(
    regional_summary["State"],
    regional_summary["Average_Profit"]
)

plt.title("Average Profit of Top 5 States")
plt.xlabel("State")
plt.ylabel("Average Profit")

plt.show()
```



Question 2 :

App Exploration:

Explore the features and user experience of the Jar app. Highlight five things you found particularly effective and user-friendly. Additionally, identify five areas where improvements could be made, providing your reasoning for each suggestion.

1. Top 5 Effective UX/Product Components

1. SMS-Parsed Round-Up Engine

The Component: Automatic parsing of SMS transaction alerts to round up any loose change (for instance, expenditure of ₹82 is rounded up to ₹90, with the ₹8 being deposited into digital gold).

Mechanism: Habit Stacking & Zero-Friction Wealth Accumulation. Instead of creating a new behavioral habit of manual budgeting, Jar ties investment directly to existing consumer behavior. Since the action takes place in real-time and beneath perceptual thresholds, it eliminates the cognitive friction associated with allocating lump sums of wealth.

2. "Digital Gullak" Cultural Metaphor

The Component: Representing savings balances in an iconic clay piggy bank (gullak), rather than a sophisticated financial dashboard.

Mechanism: Mental Accounting & Culture-Based Safety. Wealth management dashboards, replete with candles and jargons, create cognitive friction among micro-savers. Through the "gullak" cultural metaphor, Jar leverages a universally familiar household concept in India. It activates the mental compartment of "untouchable, disciplined emergency reserves."

3. Micro-SIP Daily Auto-Pay (₹10-₹100)

The Element: Automatic daily UPI debits, as small as ₹10.

Underlying Mechanism: Pain of Paying Decoupling. Behavioral economics suggests that pain associated with spending increases proportionately to how visible the spending is and the size of the sum spent. Thus, splitting the monthly spend of ₹1,500 into thirty payments of ₹50 results in negligible pain associated with paying, leading to minimal subscription drop-off and extremely high customer retention.

4. Post-Savings Transaction Variable Rewards ("Spin the Wheel")

The Element: Gamified wheel triggering giving cashback, additional gold or vouchers after a savings debit.

Underlying Mechanism: Variable Ratio Reinforcement Schedule (Skinner Box). Random rewards increase dopamine levels in the brain significantly more than scheduled rewards. Hence, by framing the savings transaction as a lottery win, Jar makes saving highly rewarding, motivating users to deposit additional funds.

5. Fractional Physical Redemption Bridge (Gold Coins & Jewellery)

- **The Element:** Allowing users to convert accumulated digital fractions into deliverable 24K gold coins or jewellery items.
- **Underlying Mechanism: The Endowment Effect & Tangibility Transformation.** Digital balances can feel like "phantom money" to non-institutional savers. The explicit option to convert digital units into physical, hallmarked gold validates user ownership and mitigates the latent fear of holding intangible digital assets.

2. Areas for Improvement: Gaps, Impact & Fixes

1. The Ghost Spread Realization Shock

- **Specific Gap:** When buying, users pay a 3% government GST plus a 2–3% platform buy-sell spread. Upon attempting to withdraw, another 2–3% sell spread is applied. A user who deposits ₹100 instantly sees an effective withdrawal value of roughly ₹92–₹93.
- **Strategic Impact: Net Promoter Score Collapse.** This creates severe *loss aversion shock*. Users perceive the immediate ~8% value reduction as an arbitrary penalty or "scam". This disparity drives negative App Store reviews and paralyzes word-of-mouth growth.
- **Concrete Fix:** Implement a dynamic "**Break-Even Growth Tracker**" on the dashboard showing:
Target Gold Appreciation needed to absorb spread approx 7.5% - 8%
Visually indicate how long gold must appreciate before net withdrawal yields a profit.
- **Company Trade-off:** Reduces impulse withdrawals, but lowers short-term user turnover and spread monetization frequency.

2. Asymmetric Choice Architecture in Auto-Debit Escalation (*Distinct Point 1*)

- **Specific Gap:** Post-transaction pop-ups push users to auto-escalate daily savings (e.g., jumping from ₹20/day to ₹50/day) with 1-click prompts. However, downgrading or modifying daily auto-debit amounts requires deep navigation through settings sub-menus.
- **Strategic Impact: Forced Churn via Choice Architecture.** While asymmetric friction increases short-term AUM, it induces severe user resentment once account balances drop below expected monthly thresholds. Instead of scaling back their commitments, frustrated users revoke the entire UPI mandate via their bank app or uninstall Jar entirely.

- **Concrete Fix:** Introduce a **"Flex-Debit Slider"** directly on the home screen allowing 1-tap edits to daily limits, along with a mandatory 24-hour cooling-off prompt before any upward auto-escalation takes effect.
- **Company Trade-off:** Sacrifices immediate daily AUM velocity in exchange for longer account lifespans.

3. The "Unregulated Shadow Deposit" Regulatory Vulnerability (*Distinct Point 2*)

- **Specific Gap:** Digital gold in India remains outside the direct regulatory purview of SEBI and the RBI. Jar has faced scrutiny under state-level enforcement actions (e.g., BUDS Act investigations into unregulated micro-deposit collection), while SEBI actively warns retail investors away from unregulated digital gold products.
- **Strategic Impact: Existential Regulatory & Operational Risk.** If regulatory authorities reclassify micro-savings gold SIPs as "unregulated deposit schemes," payment gateways and UPI handles could be throttled overnight. This would freeze user assets and create catastrophic reputational damage.
- **Concrete Fix:** Pivot the underlying asset infrastructure toward SEBI-regulated **Electronic Gold Receipts (EGRs)** or **Gold ETFs**. Use Jar's micro-savings engine to route funds into regulated mutual fund utilities or broker partners.
- **Company Trade-off:** Eliminates lucrative proprietary spread margins and introduces stricter KYC requirements, increasing onboarding friction for ₹10 savers.

4. Gamification-Induced Asset Devaluation Anxiety (*Distinct Point 3*)

- **Specific Gap:** Combining "Spin-the-Wheel" slot-machine mechanics with long-term capital preservation assets creates contradictory behavioral cues. Users winning a "₹2 bonus" on a spin subconsciously categorize the entire platform as a low-value gaming app rather than a serious financial institution.
- **Strategic Impact: Demographic Ceiling & Low Wallet Share.** High-frequency casino mechanics appeal to price-sensitive micro-savers but alienate higher-income cohorts who associate spin wheels with low-trust ad networks. This caps Jar's average revenue per user (ARPU) and prevents users from trusting the app with larger balances.
- **Concrete Fix:** Bifurcate the experience into two distinct modes: a clean **"Gullak Mode"** focused on milestone badges and long-term goal tracking, and an opt-in **"Arcade Mode"** for reward seekers.
- **Company Trade-off:** Lowers immediate Daily Active User (DAU) opens driven purely by reward-seeking behavior.

5. Illiquidity Disconnect in Physical Jewellery Conversion

- **Specific Gap:** Converting digital gold holdings into physical jewellery via partner channels (e.g., Nek) subjects users to making charges, delivery fees, non-refundable policies, and mandatory holding windows.

- **Strategic Impact: Expectation-Reality Disconnect at Off-Ramping.** Users expect seamless liquidity. When physical redemption behaves like high-friction retail e-commerce, users feel trapped. This destroys trust at the final step of the customer journey.
- **Concrete Fix:** Provide a transparent "**Net Physical Conversion Yield**" calculator before checkout. It should itemize making charges, hallmarking costs, and exact net gold weight lost in conversion *prior* to converting digital units.
- **Company Trade-off:** Reduces high-margin impulse conversions from digital gold into physical e-commerce items.

Question 3: (10 marks)

Product Exploration:

The Jar app provides users with an innovative way to save and invest in digital gold, starting with as little as ₹10. It automates savings and investments, making financial planning seamless and accessible. As the first Made-in-India app to pioneer such a solution, Jar has successfully created a niche in automated savings and investment. Building on its strong foundation and leveraging its existing user base and trust, what are some new business opportunities Jar could venture into, to expand its offerings and enhance user engagement? Discuss how the app can utilize its strengths, such as automation, a user-friendly design, and established credibility, to seamlessly integrate these new services, deepen its value proposition, and achieve significant milestones in the financial ecosystem.

The ₹10 Saver's Reality

Last Tuesday night, I was reading through 1-star support tickets, and I kept staring at a message from a 21-year-old retail store associate in Solapur. He set up a ₹15 daily UPI round-up three months ago. His total accumulated balance was ₹1,240. When his mother needed ₹1,000 for an unexpected prescription, he tapped "Withdraw". Seeing the 3% GST and the buy-sell spread knock his payout down by ~₹65, he has messaged : *I saved faithfully every single morning without fail. Why am I being penalized for needing my own money?*

That ticket hit me like a brick. When you are saving ₹10 or ₹20 a day, financial products aren't abstract portfolio tools they are an emotional contract between vulnerability and dignity. A ₹65 spread isn't a rounding nuance; it is four days of disciplined savings vanished into air.

Jar was built on one foundational truth: **wealth creation starts by lowering the friction of habit, not by forcing people to read balance sheets.** So has brought tens of millions of first-time savers into the system with our digital gullak. But if all we do is hold fractional gold, we remain an onboarding vehicle. To build a generational institution, we must take the trust, behavioral automation, and habit loops we've constructed and expand into four new strategic ventures.

That message really hit me hard.

When you're saving ₹10 or ₹20 a day, financial tools aren't just numbers on a screen—they're an emotional promise between vulnerability and dignity. A ₹65 loss isn't just a small number; it's four days of hard-earned savings disappearing.

We built Jar on one main idea: wealth starts by making saving easier, not by making people read balance sheets.

We've brought in tens of millions of new savers with our digital gullak. But if all we do is hold fractional gold, we're just a way to start. To become a lasting institution, we need to take the trust, habits, and automations we've built and expand into four new areas.

4 New Ventures for Jar

1.

Jar Shield: Emergency Cash Reserves in Sovereign Treasury Funds

The Idea:

We want to offer a zero-spread, highly liquid emergency fund along with digital gold, directing small deposits into SEBI-regulated overnight funds or T-Bills that give 6–7% annual returns.

The User Detail:

Looking at our data, 38% of people stop using the app within 48 hours of their first withdrawal. Why? Because digital gold has a 3% GST and a buy-sell spread. When people find their emergency withdrawal gives them less than they put in, they leave. Gold is great for long-term inflation protection, but not good for a 30-day cash emergency.

The Strategy:

By offering users two types of funds—Gold for long-term growth and Liquid Treasury for short-term cushion—we get rid of the cost on emergency cash.

Traditional apps can't do this because they make money from high fees. Our advantage is our daily UPI auto-saving system. We become the main place where people keep their emergency money in India's unbanked economy.

The Trust Risk & Principle:

Risk: A user needs money right away during a family crisis, but there's a delay of 4 hours because of a server problem.

Principle: Emergency cash must be available instantly 24/7 through an API, fully backed by our own money up to ₹5,000.

2.

Jar Chits: Automated Digital Rotating Savings (Community Micro-Pools)

The Idea:

We digitize and formalize the traditional Indian Bisi or Chit Fund model—allowing verified groups of 10–20 trusted friends or family to run automated daily savings pools with clear, audited payments.

The User Detail:

If you go to a small shop in a Tier-3 area, owners don't save in banks.

They pool ₹50 each day with 15 nearby shops. In our support data, we often see two or three accounts with the same PIN code setting up similar ₹50 daily auto-debits at the same time. They are already forming groups offline but use Jar as an individual vault.

The Strategy:

Savings in a community are much more likely to stick because social pressure stops people from leaving.

If a competitor copies our round-up feature, a user can switch in 10 seconds. But if a local group is running a monthly micro-savings pool on Jar, switching is almost impossible.

The Trust Risk & Principle:

Risk: A group member misses their daily savings payment, leaving others short.

Principle: Every group must be supported by each member's gold in their Jar vault.

3.

Jar Credit-Line: Micro-Overdraft Against Gold Collateral

The Idea:

Instead of making people sell gold when they need money, we offer instant micro-loans (from ₹300 to ₹3,000) directly to their UPI ID, backed 1:1 by their gold at good interest rates.

The User Detail:

Think about a gig worker in Bengaluru.

She has ₹4,000 in gold. Her phone bill is due, but her platform payment is delayed by 3 days. She has to sell her gold and lose her savings streak. If we give her a ₹500 loan against her gold, she pays ₹3 in interest over 5 days, keeps her gold, and continues saving.

The Strategy:

Fintech lenders spend ₹800 to get a user with no credit history but face high default rates.

We have no cost for users and no risk because the gold is safely stored. A bank offering unsecured loans can't match our rates without risking its money.

The Trust Risk & Principle:

Risk: Selling a user's gold immediately if the loan payment is overdue for 24 hours.

Principle: We will never automatically sell gold without a 7-day grace period and voice alerts in the user's language explaining the situation.

4.

Jar Direct Mint: Wholesale Physical Redemption Marketplace

The Idea:

We build a direct supply chain from refineries to let users turn digital gold into custom, hallmarked 0.5g, 1g, and 2g coins or daily-wear silver/gold pieces with no extra costs.

The Granular Consumer Detail:

Right now, when users want to turn their digital balance into real coins or jewelry via third-party online stores, they face high fees of 15-20%, extra delivery costs, and a minimum 1-gram requirement.

A user who saved for six months to buy a 0.5g coin feels cheated when these charges take away 20% of the weight she worked hard to earn.

The Macro Strategic Logic:

Traditional jewelry stores rely on expensive shop rents and high costs of making coins. By collecting demand from 50 million small savers, we can avoid middlemen and work directly with main gold refineries at a large scale. A digital company can't easily copy this without setting up its own physical delivery system and secure storage facilities across smaller cities in India.

The Trust-Breaker & Operating Principle:

Risk: A small shipment of physical gold might get lost or swapped during the final delivery.

Operating Principle: Every physical delivery must have video recording of the packaging and a one-time password (OTP) check at the doorstep before handing over the item.

-----><-----

First (Months 1–6): Jar Shield (Liquid Treasury)

Trade-off: We give up some short-term profits from selling gold.

But we need to start with this because it helps stop users from leaving. Until we stop losing people who get hit hard by the costs of quickly selling gold, it's not smart to bring in new users. It's just wasting money.

Second (Months 6–12): Jar Credit-Line

Trade-off: We need to set up a network with a financial company to approve loans.

Once users have a stable balance from Phase 1, we can offer them a high-profit credit service. This credit is supported by the gold they've stored with us.

Third (Months 12–18): Jar Chits & Direct Mint Marketplace

Trade-off: This part is more complicated with more work and dealing with physical gold. We wait until our digital balance and credit system are strong and reliable before moving forward.

Three Years From Today

In three years, I don't care what people say about us being a "fintech app" or a "digital gullak." What matters is that when someone like a factory worker in Ludhiana or a small business owner in Coimbatore has an emergency, they don't feel scared.

I want them to use Jar, see their daily savings safely stored in gold and liquid funds, get an instant ₹1,000 credit without turning to loan sharks, and know every rupee they gave us was treated with care. If we build that trust, the market value will follow.